

## !! Guideline Updates

# UPDATES & GUIDELINE CHANGES

---

## Jul 29

- Workflow B: `solve.sh` must be entirely original - not copied or closely mirroring the oracle solution
- Workflow B: solution must fully solve the task per `instruction.md` and produce all expected outputs; incomplete solves will be rejected

## Jul 28

- `instruction.md` must list required metric outputs and pass thresholds (exact metric names, output file/schema), aligned with `human_scores.json`, `pass_criteria`, and evaluator
- Guideline updates: clearer direction to prioritize task structure/verifier quality in review
- Instructions must specify output path/schema, target metric, and threshold for anything verifiers check
- Verifiers must be rigorous, deterministic, hard to game, but not fragile/tied to one implementation
- Verifier quality added as a High-severity checklist item for reviewers

## Jul 27

- All net-new Workflow A submissions must require GPU-based ML work; CPU-only tasks will be rejected during review

## Jul 25

- Published ["Project Otter Reviewer Quick Guide"](#) - submission checklist (Workflow A & B), common mistakes, and reviewer perspective on what gets checked before approval

## Jul 24

- Must verify oracle runs locally (`stb harbor run -a oracle -p path-to-task`) before submitting - no submitting first and waiting on platform check
- Workflow B: only `/solution` and `human_scores.json` may be edited; edits elsewhere will cause rejection

## Jul 23

- `human_scores.json` metrics must use a nested `"metrics": {...}` object (not flattened); each value numeric and must exactly match a `checks.py` key
- Checker updated to accept nested format; anyone who flattened metrics as a workaround should revert and re-run static check

## Jul 22

- Tests must check only what the instruction states, be un-gameable, and not rely on agent-visible answer files or agent-generated labels as ground truth
- `rubric_score.py`, `pass_fail.py`, `test.sh` must remain untouched skeleton files
- `task.toml` domain/subdomain must be one of 6 valid values, lowercase-with-dashes, matching content; also verify `timeout_sec`, `runtime_limits`, `schema_version="1.3"`, `gpu_types`, `source_*` fields

- Data must be real (no synthetic generation), no answer files in `task_inputs`, license must be permissive and consistent
- Tests must exist and be deterministic -  $\geq 1$  atomic binary criterion per milestone in `checks.py`; no subjective/LLM-judge grading
- Oracle must run in-container on target GPU, fully seeded, no mock/answer-reading path, thresholds set from the real run
- Dependencies must be hash-pinned/lockfile-installed; apt versions and base image digest pinned too
- Instructions must not contain solution strategies, walkthroughs, or "Graded on:" hints; ~150–200 words, no task name on line 1

## Jul 21

- `instruction.md` must specify exact output format for anything hardcoded in tests - filenames, field/column names, structure, value types
- Exact output requirements are allowed and not considered solution hints, as long as they describe *what* must be produced, not *how*

# Overview

# Otter: Expert Contributor

## Instructions

*ML coding benchmark dataset*

---

As an Expert Contributor (EC), you'll author **genuinely difficult**, end-to-end ML tasks designed to stretch frontier models across the full ML lifecycle, from data collection through final results.

**We're looking for tasks that push the strongest current models.**

### What You're Building

Each Otter task is a self-contained, single-container challenge that an AI agent must complete end-to-end - a hard ML research/engineering problem, the kind of thing a skilled engineer might have trouble with. Each task has:

- A reproducible Docker environment
- Instructions for the agent
- A golden solution

- Milestone-based tests that assess all task outputs after completion and calculate the agent's Rubric Score and Pass/Fail Grade
- Human attempts (done by other ECs) that anchor the final Pass/Fail bar for AI agents

Tasks must run unattended in a single-container, single-GPU Harbor sandbox in **4 hours or less**. All tasks must be machine learning problems **that require GPUs**.

### Task structure:

```
task_name/  
├── instruction.md  
├── task.toml  
├── solution/  
│   └── solve.sh  
├── environment/  
│   ├── task_inputs/  
│   └── Dockerfile  
├── tests/  
│   ├── test.sh  
│   ├── checks.py  
│   ├── rubric_score.py  
│   ├── pass_fail.py  
│   ├── milestones_and_rubrics.json  
│   └── human_scores.json  
└── [optional] task-level README.md
```

## What Makes a Great Otter Task

The dataset's purpose is to push frontier models on realistic, long-horizon ML work. A great task makes a strong frontier model struggle - most

attempts should fail - and it should encompass the ML workflow end-to-end. **We heavily lean toward harder tasks:** the dataset is more valuable when frontier models actually have to work for the win, so when in doubt, push the difficulty up.

### The agent should have to:

**Collect or preprocess real data.** Real data has missing values, label noise, distribution issues, and structural quirks that matter.

**Train a model under realistic constraints.** Compute and memory budgets that force genuine engineering decisions.

**Iterate on architecture or training routines.** The agent should have to make and test choices, not just run a default recipe.

### Difficulty should come from real ML challenge, not artificial friction:

The data has quirks that matter, and multiple reasonable approaches exist

The performance bar is set so strong attempts clear it and weak ones can't

The difficulty is end-to-end - a failure at any stage costs the whole task

What it should **not** come from: nitpicky output formats, pages of detailed requirements, obscure edge cases, padding for length, or trick prompts.

The test: if a strong human engineer would also fail because of the same friction, it's the wrong kind of hard - a good task should feel hard to a frontier model the same way it would feel hard to a competent human.

## Calibration and difficulty tiers

A good signal you've calibrated correctly: you can explain every pass or fail in terms of the ML (bad architectural choices, missed subtleties in the data, weak training setup) rather than prompt confusion or environment issues.

Once a task is finalized, difficulty is assigned automatically from frontier model evaluation over **k=3 runs**, using **Gemini-3.5 Flash (reasoning: high)** with the Gemini-CLI agent as the evaluation model:

**Frontier** -  $\text{Pass}@3 \leq 33\%$

**Advanced** -  $33\% < \text{Pass}@3 \leq 66\%$

**Core** -  $\text{Pass}@3 > 66\%$

Mean Pass Rate (at k=3) sets the tier. This is calculated for you, not something you fill in - use it as a target when planning a task's difficulty, not a field to set.

## Distribution and sourcing

No single domain may exceed ~30% of the total dataset; no single subdomain may exceed ~25% of the data within its domain. Submissions may be rejected if these caps are already met.

Tasks must be sourced from: (1) non-Kaggle-based complex competitions, (2) published benchmark results on academic datasets, or (3) originally-authored tasks with multiple corresponding human attempts. Simple Kaggle-style reproductions are not accepted.

## Two Workflows

Other tasks are built in two stages, and as an EC you'll be assigned to both.

### Workflow A: Task Creation

You build the full task from scratch: the instruction (`instruction.md`), an oracle solution (`solution/solve.sh`), the environment (`Dockerfile`, `task_inputs/`), the metadata (`task.toml`), and the full verifier setup (`tests/checks.py`, `tests/rubric_score.py`, `tests/pass_fail.py`, `tests/test.sh`, `tests/milestones_and_rubrics.json`, `tests/human_scores.json`). Once you submit, the task goes through review and then moves to Workflow B.

### Workflow B: Human Attempt

You're given an existing, reviewed task. Your job is to solve it as a skilled human would, without LLM assistance, and submit your `solution/solve.sh`. Each task collects multiple human attempts, which populate `tests/human_scores.json` and calibrate the task's primary metric thresholds. You don't author the instruction, environment, tests, rubric, or metadata in this workflow - you receive them, solve the task, and submit your work.

# Set Up

# Setup & Installation

---

## Prerequisites

- Docker installed and running
- uv package manager
- Python 3.12 or 3.13 (required by Harbor)

### 1. Install uv

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

### 2. Install stb

```
uv tool upgrade snorkelai-stb \  
  --find-links \  
  https://snorkel-python-wheels.s3.us-west-2.amazonaws.com/stb/index.html \  
  --python ">=3.12"
```

The `stb` command will be available globally (restart your terminal if needed).

### 3. Verify installation

```
stb --version  
stb --help
```

## 4. Authenticate

```
stb login
```

This opens the Experts platform and shows the API key dialog. Click "Generate Key," copy it, paste it into your terminal.

## 5. Configure your API keys

Export your Portkey credentials (these are the same key for both):

```
export OPENAI_API_KEY="your-portkey-api-key"
export OPENAI_BASE_URL=https://api.portkey.ai/v1
export ANTHROPIC_API_KEY="your-portkey-api-key"
export ANTHROPIC_BASE_URL=https://api.portkey.ai/v1
```

If a key is exhausted or expired:

```
stb keys refresh
stb keys show
```

## 6. Create a Modal account

Modal provides the GPU sandbox your tasks run in.

- Sign up at [modal.com](https://modal.com) with any email
- Snorkel reimburses up to \$300 in Modal usage - submit through your normal expense process with your Modal billing receipts

## 7. Install Modal

```
python3 -m pip install modal
```

## 8. Create a Modal token

```
python3 -m modal token new
```

This opens a browser window to authenticate, then writes credentials to `~/.modal.toml`; Harbor reads it automatically. On a remote machine without a browser, set environment variables instead (they take precedence over the file):

```
export MODAL_TOKEN_ID=<your-token-id>  
export MODAL_TOKEN_SECRET=<your-token-secret>
```

Grab these values from [modal.com/settings/tokens](https://modal.com/settings/tokens).

# **Workflow A: Task Creation**

# Workflow A: Creating a Task

*Workflow at a glance: Plan → Initialize → Set Up Foundation → Build the Task → Test Locally → Submit*

**Review priority:** Spend most review time on task structure and verifier quality. Confirm the task is complete, internally consistent, and meaningfully evaluated before focusing on lower-impact formatting issues.

---

## 1. Plan your task

Before you write any code, think through the task end-to-end.

- **Start with the idea.** What ML problem does this task pose? Why is it interesting? Where does it sit on the difficulty curve? See *What Makes a Great Otter Task*.
- **Break it into milestones.** A good task covers meaningful phases of the ML lifecycle (e.g. preprocessing, training, iteration). Each milestone should be a coherent, verifiable unit of work.
- **Sketch the boundaries.** For each milestone, write down the intermediate output the agent must produce and the criteria/weights for grading it.

## 2. Initialize your task

Download the task skeleton (see *Resources*) - it contains the file tree.

## 3. Set up the foundation

Get the task-level pieces in place before building out the rest:

- `task.toml` - domain, subdomain, tags, `runtime_limits` ( $\leq 4$  hours). See *Task Metadata*.
- `environment/Dockerfile` and `environment/task_inputs/` - the sandbox and its bundled data. See *Environment*.

## 4. Build the task

Build out the remaining pieces - the task is a single unified problem with milestones defined in the rubric:

- `instruction.md` - the agent's prompt. See *Instruction Prompt Styling*.
- `solution/solve.sh` - your reference solution. See *Oracle Solution*.
- `tests/checks.py` - task-specific verification logic. See *Tests*.
- `tests/milestones_and_rubrics.json` - milestones, criteria, weights. See *Milestones and Rubrics*.
- `tests/human_scores.json` - primary metric thresholds and human attempt data. See *Human Scores*.

## 5. Test locally

```
stb harbor tasks start-env -p ./my-task-name -i
```

Run your solution to confirm it passes cleanly end-to-end:

```
stb harbor run -a oracle -e modal -p ./my-task-name
```

Then test against the official evaluation model:

```
stb harbor run \  
  -m openai/@gemini/gemini-3.5-flash \  
  -e modal \  
  -p <path-to-task> \  
  --agent-kwarg disallow_web_tools=true
```

Keep iterating until the oracle solution passes deterministically every run and scores better than the model runs. See *Testing with Agents* for debugging guidance.

## 6. Review against the Submission Checklist

Before submitting, check your task against the entirety of the *Submission Checklist*. This is what reviewers use to grade your work - adhering to it greatly increases your chances of being promoted to reviewer.

## 7. Submit

When the task is clean and reproducible, submit it on the platform.

# Instruction Styling

# Instruction Prompt Styling

---

The `instruction.md` file is the primary interface between the task and the AI agent. To make Otter reflect real-world usage, instructions should feel realistic to how people actually prompt terminal agents, Claude Code, Cursor, or similar tools.

The guidelines below are intentionally somewhat loose so the dataset does not become repetitive. Lean toward using your own voice, and vary your style from task to task.

Examples of instructions we do not want are included at the bottom of this page, along with explanations of why those styles should be avoided.

## Core Philosophy

Some engineers write structured, well-formatted prompts. Others are more casual and give a few vague sentences. We want to capture that full range of realistic prompting styles.

The goal is to give the agent the what, meaning the requirements, but not the how, meaning answers, hints, or step-by-step solution guidance.

The main challenge is creating realistic variance in style while still following the five core principles below.

**Important: Prompts should not sound LLM-generated.** Avoid the “GPT-style” of writing: repetitive, overly polished, and overly polite.

## Requirements

Every `instruction.md` should follow these seven principles.

### *1. Task Instructions Must Be Well Specified*

Even though instructions should be concise, they still need to be well specified. The task goal should be clear and obvious to both a human and an agent.

All required deliverables and verifier-enforced output contracts must be stated in `instruction.md`. If the task requires a structured output, **specify the exact required file path, field or column names, nesting or table structure, expected value types, and any required wrapper or record format.**

**A useful rule of thumb is:** if a file name, field name, column name, function name, or output structure is hardcoded in `tests/checks.py`, that requirement must also appear in `instruction.md`. The agent should not be expected to guess an interface that is required only by the tests.

Describing the required output contract is not considered a solution hint. It tells the agent what must be produced without prescribing how to produce it.

Watch out for tasks that are mostly difficult because they contain too many edge cases or requirements. If a task is hard mainly because it has a large number of detailed conditions, it should be rejected.

## ***2. Task Instructions Must Be Interesting***

The task should not be so obscure or irrelevant that no meaningful group of developers or users would find it useful.

Every task should be interesting, practical, or valuable in some way.

## ***3. Task Instructions Must Not Give Answers or Hints***

These tasks should represent one-shot requests from a user to a terminal agent.

If the `instruction.md` includes significant hints, rubrics, or guidance on how to solve the task, it is not representative of the style we want.

Requirements are fine. Step-by-step solution guidance is not.

## ***4. Task Instructions Must Not Be Over-Prescribed***

Instructions should give the agent room to reason.

A good `instruction.md` reads like a real user request — it describes what's needed, not how to get there. Avoid step-by-step workflows, implementation hints, or guidance that narrows the solution path before the agent has a chance to think.

## 5. Task Instructions Must Be Unique

The task must be noticeably different from tasks in other harbor-style projects.

A task is usually too similar if neither of the following is true:

- The initial state or instructions are different in a non-trivial way
- The expected output is different in a non-trivial way

## 6. Task Instructions Must Use Absolute Paths

When referencing files or directories, use absolute paths.

## 7. Task Instructions Must be Realistic

The instructions should have real-world use and be natural-sounding. They should reflect how you would normally interact with an LLM.

# Human-Centric vs. Synthetic Styling

Use this comparison as a guide:

| Category | Avoid                                              | Prefer                                                |
|----------|----------------------------------------------------|-------------------------------------------------------|
| Tone     | "You are an expert programmer. Your goal is to..." | "We need to migrate the existing SQLite schema to..." |
| Length   | 500+ words of redundant context                    | 150–200 words of actionable information               |

|          |                                                                |                                                                      |
|----------|----------------------------------------------------------------|----------------------------------------------------------------------|
| Guidance | “First, use the <code>ls</code> command to see files, then...” | “The source data is in <code>/data</code> . Output the result to...” |
|----------|----------------------------------------------------------------|----------------------------------------------------------------------|

## Common Prompting Errors and Examples

Below are examples of instructions that violate the core principle: prompts should sound human and give the agent the what, not the how.

Each section includes a real example and an explanation of why it fails.

### 1. Step-by-Step Walkthrough With Solution Values

#### Example

None

##### Step 3: Optimize the File Sync Utility

Modify `/app/file_sync.py` to implement the following optimizations:

1. Socket Buffer Tuning
  - Set `SO_RCVBUF` to `262144` bytes, or 256 KB
  - Set `SO_SNDBUF` to `262144` bytes, or 256 KB
  - Enable `SO_REUSEADDR`
2. TCP Settings
  - Disable `TCP_NODELAY`, set to `0`, to allow Nagle's algorithm for better bulk transfer
3. Buffer Size Optimization
  - Use `65536` byte, or 64 KB, chunks for file reading and socket sending

#### Why it's bad

The prompt tells the agent exactly what to do, which defeats the purpose of the project. We should not give the agent an explicit walkthrough for solving the problem.

## 2. Hints Section That Describes the Answers

### Example

None

##### Detection Guidance

Look for:

- Currency conversion anomalies that affect only specific currency codes
- Date field values that are valid but transposed relative to the source
- Tax calculations where component ratios vary by product category
- Cross-source matching that incorrectly conflates temporally distant records
- Running balance computations that skip records at period boundaries
- Display formatting precision that varies by value magnitude

### Why it's bad

This gives away the answers. The section explains exactly how to detect each type of corruption.

## 3. Excessive Markdown or Bulleted Structure

### Example

None

##### `revenue-by-category` `GET`

Array sorted by `totalRevenue` descending. Fields: `category`, `totalRevenue`, calculated as the sum of `quantity \* unitPrice` rounded to two decimal places, `orderCount`, and so on.

##### `top-customers` `GET`

Array sorted by `totalSpent` descending. Optional `?limit=N`, default 5. Fields: `customerId`, `customerName`, and so on.

##### `order-status-summary` `GET`

Array sorted by `count` descending and covering all statuses. Fields: `status`, `count`, `percentage`, and so on.

### Why it's bad

This reads like structured documentation, not a human prompt. Avoid excessive markdown and long bullet lists.

## 4. Overly Prescriptive Guidelines

### Example

None

`backend/processor.py` must export four functions:

- `apply\_grayscale(input\_path: Path, output\_path: Path) -> Path`
- `apply\_rotate(input\_path: Path, output\_path: Path, angle: float) -> Path`
- `apply\_resize(input\_path: Path, output\_path: Path, width: int, height: int) -> Path`
- `get\_image\_dimensions(input\_path: Path) -> tuple[int, int]`

### Why it's bad

This tells the agent the exact project structure, exact functions, expected signatures, and implementation shape. It is too prescriptive.

## 5. Bold Markers Highlighting Solution Details

### Example

None

**\*\*Policy limit:\*\*** \$500 USD per single expense.

**\*\*Note:\*\*** Policy violations, meaning expenses over \$500, operate independently from the approval system. They do not prevent an expense from also being evaluated and counted in the approval statistics.

**\*\*Compliance Rate Definition:\*\***  $1.0 - (\text{violation\_count} / \text{total\_expenses})$

### Why it's bad

Bold markers draw attention to exact solution details. This turns requirements into hints and makes the task less realistic.

# Task Metadata

# Task Metadata (task.toml)

---

Every task has a task.toml file that captures its metadata. This is what makes the task discoverable, properly categorized, and correctly configured for evaluation.

For Workflow A (Task Creation), you must include the `dataset_license` field under the `[source]` block. It must specify a permissive license (e.g., MIT, Apache-2.0, CC-BY-4.0).

## Fields you fill in

| Field             | Description                                                                                                                  |
|-------------------|------------------------------------------------------------------------------------------------------------------------------|
| runtime_limits    | Maximum agent runtime for the task, in seconds. Must be no greater than 4 hours (14400).                                     |
| schema_version    | Must be a string. Should be "1.3"                                                                                            |
| <b>[task]</b>     |                                                                                                                              |
| name              | Should be "otter/task-name". Task name should be in kebab case.                                                              |
| description       | Short summary of what the agent is asked to build or do.                                                                     |
| <b>[metadata]</b> |                                                                                                                              |
| domain            | One of the six domains in the table below.                                                                                   |
| subdomain         | A subdomain that fits within your chosen domain (see table).                                                                 |
| tags              | 3 to 6 ML keywords describing relevant techniques, frameworks, or libraries (e.g., pytorch, transformer, LoRA, time-series). |

|                                |                                                                                                                                                                                                  |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| source_dataset (optional)      | Name of the dataset the task is built from.                                                                                                                                                      |
| Source_url (optional)          | Link to the dataset source.                                                                                                                                                                      |
| Source_reference_id (optional) | Citation or DOI for the source dataset.                                                                                                                                                          |
| Source_license (optional)      | License under which the source dataset is distributed (must permit your use). Allowed are: MIT, BSD, BSD-2-Clause, BSD-3-Clause, Apache-2.0, Apache 2.0, CC0, CC0 1.0, CC-BY-4.0, CC BY 4.0, ISC |
| <b>[environment]</b>           |                                                                                                                                                                                                  |
| build_timeout_sec              | Max time allowed to build the environment, in seconds.                                                                                                                                           |
| workdir                        | Working directory inside the container.                                                                                                                                                          |
| os                             | Operating system of the environment.                                                                                                                                                             |
| cpus                           | Number of CPU cores allocated.                                                                                                                                                                   |
| memory_mb                      | RAM allocated, in MB.                                                                                                                                                                            |
| storage_mb                     | Disk storage allocated, in MB.                                                                                                                                                                   |
| gpus                           | Number of GPUs allocated. Must be > 0.                                                                                                                                                           |
| gpu_types                      | GPUs used to run the task. (e.g. gpu_types = ["H100", "A100"])                                                                                                                                   |
| allow_internet                 | Must be <b>true</b> . Required to enhance the realism of frontier difficulty evaluations.                                                                                                        |
| <b>[agent]</b>                 |                                                                                                                                                                                                  |

|                   |                                                                                      |
|-------------------|--------------------------------------------------------------------------------------|
| timeout_sec       | Max wall-clock time the agent has to complete the task. Should match runtime_limits. |
| <b>[verifier]</b> |                                                                                      |
| timeout_sec       | Max time the verifier has to run and score the solution, in seconds.                 |

The maximum agent runtime must be no greater than 4 hours.

## Domains and subdomains

Classify the task with at least one domain, subdomain, and tag.

| Domain               | Subdomains                                                                                                                                                            |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| tabular-ml           | classification, regression, event-production, anomaly-detection, clustering, recommender-systems                                                                      |
| computer-vision      | image-classification, object-detection-tracking, image-segmentation                                                                                                   |
| nlp                  | sentiment-analysis, information-extraction, question-answering, lm-fine-tuning                                                                                        |
| audio-learning       | audio-classification, speech-recognition, sound-event-detection, audio-enhancement-denoising                                                                          |
| multi-modal-learning | 3d-scene-classification-analysis, image-text-text-image, multi-modal-feature-fusion                                                                                   |
| graph-learning       | node-classification, node-regression, node-clustering, edge-classification, edge-prediction, graph-classification, graph-regression, graph-matching, graph-generation |

## Example task.toml

```
Shell
runtime_limits = 14400
schema_version = "1.3"

[task]
name = "otter/robust-ai-image_detection-deepshield"
description = "Build a robust AI-generated image detector for DeepShield
social-platform distribution shift."

[metadata]
domain = "computer-vision"
subdomain = "object-detection-tracking"
tags = [
    "real-ml",
    "computer-vision",
    "ai-image-detection",
    "social-platform-shift",
    "deepshield",
    "robustness",
]
source_dataset = "DeepShield: A Dataset of Real and AI-Generated Images also
Shared on Social Networks"
source_url = "https://zenodo.org/records/15648378"
source_reference_id = "10.5281/zenodo.15648378"
source_license = "CC-BY-4.0"

[environment]
build_timeout_sec = 900.0
workdir = "/app"
os = "linux"
cpus = 4
memory_mb = 8192
storage_mb = 20480
gpus = 1
gpu_types = ["A100"]
allow_internet = true

[agent]
timeout_sec = 14400.0
```

```
[verifier]
timeout_sec = 600.0
```

## Tagging guidelines

- Be specific. lora is more useful than fine-tuning. graph-attention-network beats gnn.
- Include the framework when it matters (pytorch, jax, tensorflow).
- Don't pad to hit 6. Three well-chosen tags beat six vague ones.
- No marketing language. Skip state-of-the-art, cutting-edge, etc.

## Setting runtime\_limits

The `runtime_limits` field in `task.toml` caps the maximum agent runtime for the task. Set it as a generous upper bound, not based on your oracle's runtime; agents will take longer than your reference solution. The maximum agent runtime must be no greater than 4 hours to ensure tasks are bounded and sufficiently reproducible.

## Common mistakes

- Setting `runtime_limits` based on your oracle's runtime. Set it as a generous upper bound while staying under the 4-hour cap.
- Choosing a domain or subdomain classification that doesn't actually match the task, such as calling something segmentation when it's really classification with masks.

# Environment

# Authoring an Environment (Dockerfile + Task Inputs)

---

The `environment/` directory sets up the sandbox your task runs in. It contains two things: the `Dockerfile` that builds the container, and `task_inputs/` which holds any data, checkpoints, or other files the agent will need.

The whole environment must be reproducible, lightweight, and able to run without privileged mode.

## Dockerfile

The Dockerfile must live at `environment/Dockerfile`.

## Basic Structure

None

```
# Pin the base image. Digest required; see "Key Principles" below.  
FROM python:3.11-slim@sha256:<digest>
```

```
WORKDIR /app
```

```
# Install system dependencies. Pin versions.  
RUN apt-get update && apt-get install -y --no-install-recommends \  
    git=<git-version> \  
    curl=<curl-version> \  
    && rm -rf /var/lib/apt/lists/*
```

```
# Install Python dependencies. Pin versions.  
RUN pip install --no-cache-dir \  
    torch==2.3.0 \  
    transformers==4.40.0 \  
    numpy==1.26.4 \  
    pandas==2.1.0
```

```
# Set environment
ENV PYTHONPATH=/app
```

Replace the `sha256` digest with one from the registry for the image you choose. You can get it with:

```
None
docker manifest inspect <image>:<tag>
```

You can also find the digest on the image page in Docker Hub.

## Key Principles

### 1. Pin Everything

The `FROM` line is a dependency. Pin the base image with a digest using `image@sha256:....`

Floating tags, including major or minor version tags, can be rebuilt and may change system libraries without any Dockerfile edits. That breaks reproducibility.

Python packages must be installed from a lockfile or with `--require-hashes`. Exact versions alone are not sufficient for reproducible installs.

```
None
```

```
Good:
```

```
Dockerfile:
```

```
RUN pip install --require-hashes -r requirements.txt
```

```
** requirements.txt must pin exact versions and include hashes, for
example:
```

```
torch==2.3.0 \
```

```
--hash=sha256:<hash>
```

Bad:

```
RUN pip install torch
```

```
RUN pip install 'torch>=2.0'
```

```
RUN pip install torch==2.3.0
```

System packages should always be pinned.

## ***2. Never Copy the Solution or Tests***

Snorkel's harness mounts these automatically. Including them in the image is a hard fail.

None

```
# Wrong – never do this  
COPY solution/ /solution/  
COPY tests/ /tests/
```

## ***3. No Privileged Mode***

Do not require root-only behavior or privileged Docker settings. Tasks must run in standard sandbox mode.

## ***4. Keep the Image Lightweight***

Use `-slim` base images when possible.

Clean up apt cache after installs

None

```
RUN rm -rf /var/lib/apt/lists/*
```

Do not install packages the task does not actually need.

Large datasets and checkpoints should go in `environment/task_inputs/`, not baked into the image.

## Common Pattern: Requirements File

None

```
FROM python:3.11-slim@sha256:<digest>
```

```
WORKDIR /app
```

```
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

```
ENV PYTHONPATH=/app
```

If you use a `requirements.txt`, every line in it must be pinned.

None

```
torch==2.3.0
```

```
transformers==4.40.0
```

```
numpy==1.26.4
```

```
pandas==2.1.0
```

## Constraints

Use a single container and a single GPU. Multi-container setups, such as `docker-compose` with a separate database service, are not accepted. Everything the agent needs must live in one container.

Do not require privileged mode.

Keep the image under roughly 5 GB. Large model weights and datasets should go in `environment/task_inputs/`, not image layers.

## Test Your Dockerfile Locally Before Submitting

None

```
cd environment
docker build -t my-task .
docker run -it my-task bash
```

From inside the container, sanity-check that:

- Python imports your dependencies without errors
- Any binaries the oracle needs are on `PATH`

## Common Dockerfile Mistakes

- Forgetting the digest pin on `FROM`, which is the most common reproducibility issue
- Using unpinned `pip install`, which is the second most common issue
- Copying `solution/` into the image "for testing" and forgetting to remove it
- Baking a 5 GB dataset into the image instead of putting it in `task_inputs/`

## Task Inputs (`environment/task_inputs/`)

The `task_inputs/` directory holds everything the agent needs to start work that isn't a software dependency. The directory is part of the environment, so its contents are available inside the container at the paths referenced by `instruction.md`, `solution/solve.sh`, and the agent's code.

## What Belongs Here

- Datasets (CSVs, parquet, image folders, audio files, etc.)
- Pretrained model checkpoints the task expects the agent to start from
- Configuration files, schemas, or reference data the task needs
- Any source files, code stubs, or starter code the agent reads or modifies
- Any other input artifact the task references that isn't installed by the Dockerfile

## What Does Not Belong Here

- **Oracle solutions.** Those go in `solution/solve.sh`.
- **Test files, test fixtures, or expected outputs used for grading.** Those go in `tests/`.
- **Anything containing ground-truth answers** that an agent could read to short-circuit the task.
- **Software dependencies** that should be installed by the Dockerfile (libraries, system packages, etc.).

## Sourcing and Bundling Data

All task datasets must come from a real, externally sourced dataset or an approved task from the [Task Gallery](#). Synthetic, manufactured, or script-generated datasets are not allowed. Download the dataset once during task authoring and place the required files in `environment/task_inputs/`; do not fetch dataset content at build time or solve time.

## No Synthetic Data is Allowed

Make sure the data has a license that allows commercial use (e.g. MIT, Apache, CC0)!

## Common Task Inputs Mistakes

- Including ground-truth answers (labels, expected outputs, oracle artifacts) that the agent could read directly
- Bundling a 50 GB dataset when the task only uses a 200 MB subset
- Relying on a build-time download instead of bundling the data
- Including data with a restrictive license

- Putting starter code in the Dockerfile via `COPY` instead of in `task_inputs/`

# Oracle Solution

# Writing the Oracle Solution (`solution/solve.sh`)

---

The oracle solution is the top-performing human solution for the task. It is a shell script (`solution/solve.sh`) that reliably completes the task end-to-end.

The oracle lives at `solution/solve.sh` at the task root. It serves as the ground truth for milestone, rubric, and reward alignment, and must deterministically satisfy all task-specific checks in `tests/checks.py` as scored by the fixed harnesses in `tests/rubric_score.py` and `tests/pass_fail.py`. The grading of the oracle mirrors how model solutions to the task will be scored.

## What the Oracle Must Do

Each `solve.sh` must:

- Run end-to-end without intervention
- Deterministically satisfy all task-specific checks in `tests/checks.py` as scored by `tests/rubric_score.py` and `tests/pass_fail.py`
- Produce all required intermediate milestone outputs and final outputs, including preprocessing artifacts, data files, model checkpoints, metrics, and any other files referenced by `tests/checks.py` and `tests/milestones_and_rubrics.json`
- Be derived from the top-performing human attempt for the task, achieving a Pass grade against `tests/human_scores.json`

If your oracle does not pass cleanly against its own `tests/checks.py`, `tests/rubric_score.py`, and `tests/pass_fail.py`, the task is broken.

## Where the Oracle Lives

The oracle lives at::

`solution/solve.sh`

This single oracle reliably completes the entire task end-to-end, producing all intermediate milestone outputs and final outputs required by `tests/checks.py` and `tests/milestones_and_rubrics.json`.

## Basic Structure

Shell

```
#!/bin/bash
set -e # Exit on error

cd /app

# Preprocess data
python preprocess.py \
  --input /app/task_inputs/raw_data.csv \
  --output /app/workdir/processed/

# Train model
python train.py \
  --data /app/workdir/processed/ \
  --output /app/workdir/checkpoints/ \
  --seed 42

echo "Task completed successfully"
```

## Key Principles

### 1. Show the steps, not the answer

The oracle should derive the result, not hardcode it. The point is to demonstrate a real solution path that the task's tests can validate.

Shell

```
# Wrong - hardcoded answer
echo '{"accuracy": 0.94}' > /app/results/metrics.json
```

```
# Right – derived from actual computation
python generate_metrics.py \
  --checkpoint /app/workdir/checkpoints/best.pt \
  --output /app/results/metrics.json
```

## 2. Make it deterministic

Running the oracle twice must produce the same outputs. The task's checks and scoring harnesses depend on this.

Common sources of non-determinism to control:

- Python `random`
- `numpy.random`
- `torch.manual_seed`
- `tf.random.set_seed`
- CUDA non-determinism (use `torch.use_deterministic_algorithms(True)` where feasible)
- Time-dependent operations like timestamps in filenames
- External network calls (avoid by bundling data in `environment/task_inputs/`)

## 3. Produce every required output

The `instruction.md` lists the outputs the agent must produce. Your oracle must produce all of them in the right paths and formats. The task's checks and scoring harnesses will fail the oracle if any required output is missing.

## 4. Achieve a Pass grade

Your oracle must achieve a Pass grade against `tests/human_scores.json`. Meaning it surpasses the task's human scores or meets the requisite percentage of aggregate human scores on the final pass/fail milestone, and earns a high Rubric Score against `tests/milestones_and_rubrics.json`. The oracle represents a top-performing human solution, so it should clear the bar comfortably.

## 5. Fail fast

Use `set -e` at the top so any error halts the script. Silent failures are worse than loud ones.

## 6. Write the solution yourself

The oracle represents what a skilled human can do unaided. Minimal LLM assistance for syntax, such as checking Bash flags, is fine. The actual solution approach and implementation must be yours.

# Embedding Python Directly

For short Python work, heredoc syntax keeps everything in one file:

bash

```
Shell
#!/bin/bash
set -e
cd /app

python << 'EOF'
import pandas as pd

df = pd.read_csv('/app/task_inputs/data.csv')
df['total'] = df['price'] * df['quantity']
df.to_csv('/app/workdir/processed/data.csv', index=False)
EOF
```

For anything substantial, such as model training or multi-file logic, write a proper `.py` file and call it from `solve.sh`.

# Testing Your Oracle

For your oracle, confirm that:

- The script completes without errors within `runtime_limits`
- All required intermediate milestone outputs are produced in the right paths
- All required final outputs are produced in the right paths
- Each milestone's criteria pass against `tests/milestones_and_rubrics.json`
- Running it twice produces identical results

Then test the full task end-to-end:

- The oracle produces every artifact that downstream milestones depend on
- `tests/rubric_score.py` returns a high Rubric Score across all milestones
- `tests/pass_fail.py` returns a Pass grade against `tests/human_scores.json`

## Common Mistakes

- Hardcoding outputs instead of computing them
- Leaving randomness unseeded, causing the solution to pass once and fail later
- Missing outputs that `tests/checks.py` or the rubric expects
- Depending on the network during solve time instead of bundling data in `environment/task_inputs/`
- Skipping `set -e`, which can let earlier failures silently corrupt downstream work
- Writing `solve.sh` only from your own perspective and forgetting to verify it against `tests/checks.py`, `tests/rubric_score.py`, and `tests/pass_fail.py`
- Failing to produce all intermediate milestone outputs that the rubric checks, not just the final outputs

# Writing Tests

# Writing Tests

You write the test suite for your task. These tests assess all milestone outputs after the task is attempted and produce the agent's Rubric Score and Pass/Fail Grade.

You also write the rubric (`tests/milestones_and_rubrics.json`) and populate the human scores (`tests/human_scores.json`) that the Pass/Fail Grade is calculated against.

## Where Tests Live

| File                                           | What you write here                                                                                                                                                                                                                                     | What it Does                                                                                                                                                                                                                                                                     |
|------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>tests/rubric_score.py</code>             | Nothing. This is the first fixed, shared harness. Do not edit it.                                                                                                                                                                                       | It imports <code>checks.py</code> , applies criterion weights and milestone critical flags from <code>milestones_and_rubrics.json</code> , computes the Per-Milestone Scores and Overall Rubric Score, and writes the intermediate rubric result.                                |
| <code>tests/pass_fail.py</code>                | Nothing. This is the second fixed, shared harness. Do not edit it.                                                                                                                                                                                      | It reads the rubric result, compares the metrics against <code>human_scores.json</code> , generates the final pass/fail milestone, and computes the Binary Pass/Fail Grade.                                                                                                      |
| <code>tests/checks.py</code>                   | Your task-specific tests. Must expose <code>evaluate(context)</code> , which runs every check and returns criteria (per-check pass/fail) plus a metrics dict of the agent's actual measured values. <b>Do not implement threshold comparisons here.</b> | Returns criteria and metrics that <code>rubric_score.py</code> and <code>pass_fail.py</code> use to calculate the scores. Criterion ids must match <code>milestones_and_rubrics.json</code> ; metric keys must match <code>human_scores.json</code> <code>pass_criteria</code> . |
| <code>tests/milestones_and_rubrics.json</code> | Milestones and their criteria. For each criterion, set a weight and assign its weight and set the milestone's critical flag                                                                                                                             | Supplies criterion weights for the Rubric Score and the milestone-level critical flags used for Pass/Fail. A critical milestone must pass completely; non-critical milestones affect the diagnostic Rubric Score but do not independently force a Fail.                          |
| <code>tests/human_scores.json</code>           | The human attempt record(s) plus the <code>pass_criteria</code> block defining the primary metric(s), their pass thresholds, and comparison.                                                                                                            | Supplies the metric thresholds for Pass/Fail. The agent passes only <b>if all required criteria pass AND every primary metric meets its threshold.</b>                                                                                                                           |
| <code>tests/test.sh</code>                     | Nothing. This is the fixed wrapper that runs <code>rubric_score.py</code> and then <code>pass_fail.py</code> . You                                                                                                                                      | Not graded. Edit only as needed for verifier dependencies and runner configuration.                                                                                                                                                                                              |

|  |                                                                                                         |  |
|--|---------------------------------------------------------------------------------------------------------|--|
|  | must never edit it except where explicitly permitted for verifier dependencies or runner configuration. |  |
|--|---------------------------------------------------------------------------------------------------------|--|

## Build a Comprehensive Test Suite

The test suite is what makes your task evaluable. A weak suite that only checks "did the file exist" will pass any solution that produces output, including ones that solve the wrong problem.

For each milestone, your tests should cover:

- **Every explicit output the instruction asks for.** If `instruction.md` says "produce a `metrics.json` with an `f1` field," test that the file exists, parses, has the field, and that the field is a valid number in range.
- **Critical implicit requirements.** These are the things any reasonable solution must satisfy that you didn't bother to spell out in the instruction. Examples: predictions are valid probabilities, no NaNs in outputs, splits don't overlap, checkpoint isn't suspiciously small or empty, metric was computed on the right data.
- **Failure modes you'd catch in a code review.** If you'd reject a teammate's PR for shipping a model that overfits the test set or a preprocessing script that drops half the rows, your tests should catch the same thing.

A good gut check: imagine an agent that does the bare minimum to produce output without actually solving the problem. Would your tests catch it? If yes, the suite is doing its job. If no, add criteria.

Don't pad to hit a number. Each milestone requires at least 1 criterion; serious milestones will usually need more.

## Task Tests: `tests/checks.py`

Use standard python for deterministic checks. Each deterministic test function should check one programmatically verifiable thing.

## Basic Structure

Python

```
"""Deterministic tests for milestone outputs of <task-name>."""

import json
from pathlib import Path

def test_preprocessed_data_exists():
    """Preprocessed train/val/test splits exist."""
    for split in ["train", "val", "test"]:
        assert
Path(f"/app/workdir/processed/{split}.parquet").exists(), \
    f"Missing preprocessed {split} split"

def test_model_checkpoint_exists():
    """Trained model checkpoint was produced."""
    assert Path("/app/workdir/checkpoints/best.pt").exists()

def test_metrics_file_valid():
    """Final metrics file exists and parses as JSON."""
    metrics_path = Path("/app/workdir/results/metrics.json")
    assert metrics_path.exists()

    with open(metrics_path) as f:
        metrics = json.load(f)

    assert "f1" in metrics
    assert isinstance(metrics["f1"], (int, float))
    assert 0.0 <= metrics["f1"] <= 1.0
```

## Principles

## Test behavior, not implementation

Run the agent's outputs and check results. Don't grep through the agent's source code looking for patterns. That's brittle and easy to game.

Verifiers should be rigorous, deterministic, and difficult to game without being fragile. They should evaluate the correctness of the agent's outputs while allowing valid alternative solution approaches. Avoid checks tied to exact code structure, incidental formatting, wall-clock timing, or one expected implementation path.

python

Python

```
# Wrong - gameable and brittle
def test_uses_lora():
    source = open("/app/train.py").read()
    assert "LoraConfig" in source

# Right - checks the actual outcome
def test_lora_checkpoint_size():
    """LoRA-tuned checkpoint should be under 100 MB."""
    size = Path("/app/workdir/checkpoints/best.pt").stat().st_size
    assert size < 100 * 1024 * 1024
```

## Use informative docstrings

Every test should have a docstring explaining what it checks.

## Cover edge cases

Don't only test the happy path.

Python

```
def test_predictions_are_valid_probabilities():
    """All predicted probabilities are in [0, 1] and contain no
    NaN."""
```

```

import numpy as np

preds = np.load("/app/workdir/results/predictions.npy")

assert not np.isnan(preds).any()
assert (preds >= 0.0).all()
assert (preds <= 1.0).all()

```

## ***Fail loudly when outputs are missing***

Python

```

# Wrong - silently passes if file is missing
def test_metrics():
    if Path("/app/workdir/results/metrics.json").exists():
        with open("/app/workdir/results/metrics.json") as f:
            assert json.load(f)["f1"] > 0.5

# Right - fails loudly
def test_metrics():
    metrics_path = Path("/app/workdir/results/metrics.json")

    assert metrics_path.exists(), "Metrics file not produced"

    with open(metrics_path) as f:
        assert json.load(f)["f1"] > 0.5

```

## ***Keep tests aligned to milestone outputs***

Tests should check the intermediate and final outputs requested in `instruction.md` and represented in `milestones_and_rubrics.json`. Avoid testing artifacts the task never asked the agent to produce.

# Anti-Patterns to Avoid

## *Brittle exact-string matching*

Python

```
# Bad
assert output == "Training complete\n"
```

```
# Good
assert "complete" in output.lower()
```

## *Hardcoded random values*

Python

```
# Bad - assumes a specific output
assert preds[0] == 0.4287
```

```
# Good - checks properties
assert all(0.0 <= p <= 1.0 for p in preds)
```

## *Order-dependent tests*

Each test must be independent. Don't share state through globals or rely on tests running in a specific order.

## *Criteria that aren't deterministically verifiable*

If a check requires human judgment, it doesn't belong in a task test. Task criteria must be binary, atomic, and deterministic.

**tests/test.sh**

The runner sets up the test environment, runs `tests/rubric_score.py` followed by `tests/pass_fail.py`, and writes the reward file.

bash

Shell

```
#!/usr/bin/env bash
set -uo pipefail

# Standard Harbor verifier entrypoint. Do not edit this file.
#
# Invokes the two standardized scoring scripts, in order:
#   1. rubric_score.py - runs checks.py once, computes the Rubric
#   Score
#   2. pass_fail.py    - computes the final Pass/Fail Grade and writes
#                       reward.json / reward.txt for Harbor

LOG_DIR="${OTTER_LOG_DIR:-/logs/verifier}"
mkdir -p "${LOG_DIR}"

PYTHON_BIN="${PYTHON:-python3}"
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"

RUBRIC_SCORE_PY="${OTTER_RUBRIC_SCORE_PY:-/tests/rubric_score.py}"
if [[ ! -f "${RUBRIC_SCORE_PY}" ]]; then
    RUBRIC_SCORE_PY="${SCRIPT_DIR}/rubric_score.py"
fi

PASS_FAIL_PY="${OTTER_PASS_FAIL_PY:-/tests/pass_fail.py}"
if [[ ! -f "${PASS_FAIL_PY}" ]]; then
    PASS_FAIL_PY="${SCRIPT_DIR}/pass_fail.py"
fi

"${PYTHON_BIN}" "${RUBRIC_SCORE_PY}"
rubric_status=$?
```

```

# pass_fail.py always runs so a reward is always produced; it reads
# rubric_score.py's output (including any error) from
rubric_result.json.
"${PYTHON_BIN}" "${PASS_FAIL_PY}"
pass_fail_status=$?

if [[ ! -f "${LOG_DIR}/reward.txt" ]]; then
    echo 0 > "${LOG_DIR}/reward.txt"
fi

if [[ ${rubric_status} -ne 0 ]]; then
    exit "${rubric_status}"
fi
exit "${pass_fail_status}"

```

Important notes:

- Do not edit this file!
- Test dependencies should be installed in the Dockerfile
- Keep the agent runtime environment separate from the test environment
- The script must produce `/logs/verifier/reward.txt` or `/logs/verifier/reward.json`

## Validating Your Tests

Run the task-level oracle against the task tests:

```

None
stb harbor run -a oracle -p ./my-task

```

The oracle should run end-to-end, produce all required outputs, satisfy the task-specific checks in `tests/checks.py`, receive a high Rubric Score from `tests/rubric_score.py`, and receive a Pass grade from `tests/pass_fail.py` against `tests/human_scores.json`. If not, either the oracle is incomplete, the rubric is misaligned, the human scores are inconsistent with the evaluator, or the tests are broken.

Then break the oracle deliberately. Delete a required output, skip a required computation, corrupt a metric, or produce something obviously wrong. The relevant test or rubric criterion should fail. If everything still passes, your tests aren't actually checking what they claim to check.

## Common Mistakes

- Defining fewer than 2 intermediate milestones.
- Declaring a final pass/fail milestone in `milestones_and_rubrics.json`. The fixed harness generates this milestone automatically from `human_scores.json`.
- Assigning criterion weights that don't reflect the relative importance of each criterion.
- Tests that only check file existence and miss whether the contents are valid.
- Tests that pass for any output, including obviously broken ones.
- Criteria that aren't binary, atomic, or verifiable.
- Trying to grade subjective things without clear rubric criteria.
- Installing test dependencies in the Dockerfile instead of `test.sh`.
- Grepping source code instead of checking outputs.
- Defining milestone outputs in `instruction.md` that no test or rubric criterion checks.
- Letting tests, `milestones_and_rubrics.json`, and `human_scores.json` drift out of sync.

# Human Scores

# Human Scores

## (human\_scores.json)

---

### human\_scores.json

This doc explains what `human_scores.json` is and what you put in it when you submit a task in Workflow A.

#### What it is

`human_scores.json` contains the human scores necessary to calculate the model's final Pass/Fail Grade on the task. The evaluator uses these scores to determine whether an agent surpassed the task's human scores or met the required percentage of aggregate human scores.

When you submit your task in Workflow A, `human_scores.json` should contain the human score data required by the evaluator for the task. These scores are used to calculate the task's final Pass/Fail Grade.

#### Where it lives

```
None
task_name/
├── tests/
│   ├── checks.py
│   ├── rubric_score.py
│   ├── pass_fail.py
│   ├── human_scores.json
│   ├── milestones_and_rubrics.json
│   └── test.sh
```

This file lives in the task-level `tests/` directory.

## What you submit

A human score record capturing a task run using the task-level oracle in `solution/solve.sh`.

json

JSON

```
{
  "task_id": "robust_ai_image_detection_deepshield",
  "pass_criteria": {
    "accuracy": 0.64,
    "f1": 0.65,
    "max_platform_fpr_disparity": {
      "threshold": 0.25,
      "comparison": "lte"
    },
    "max_generator_fnr_disparity": {
      "threshold": 0.70,
      "comparison": "lte"
    }
  },
  "attempts": [
    {
      "id": "human_1",
      "architecture": "Logistic-style supervised classifier over handcrafted image features.",
      "augmentations": "Color, texture, RGB/HSV histogram, gradient, residual, and frequency/statistical features; class-balanced training on the clean PreSocial split (no image synthesis).",
      "description": "Logistic-style linear detector trained on a compact handcrafted image-feature table. Method: build the color/texture/histogram/gradient/residual/frequency feature table from the clean PreSocial split, fit a calibrated linear model with class-balanced weighting, select a decision threshold, and evaluate robustness on the social-platform (Facebook/Telegram/X) test split.",
      "metrics": {
        "accuracy": 0.6827956989247311,
```

```

    "f1": 0.7121951219512195,
    "max_platform_fpr_disparity": 0.0,
    "max_generator_fnr_disparity": 0.6666666666666666
  }
}

```

**Important:** `attempts[ ].metrics` must be a JSON object containing one numeric field per metric. Do not place metric values directly on the attempt record. Each metric name must exactly match a key returned in the `metrics` dictionary by `tests/checks.py`

## Field reference

| Field                                              | Description                                                                                                                                                                                                                  |
|----------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>task_id</code>                               | Identifier for the task.                                                                                                                                                                                                     |
| <code>pass_criteria</code>                         | Object mapping each metric name to its passing requirement — either a bare numeric value or a <code>{threshold, comparison}</code> object. Defaults to greater than.                                                         |
| <code>pass_criteria[ ].threshold</code>            | Numeric value. The threshold of the metric that should be used for grading.                                                                                                                                                  |
| <code>pass_criteria[ ].comparison</code>           | The operator used to compare the achieved metric value against the threshold. One of <code>lte</code> ( $\leq$ ), <code>gte</code> ( $\geq$ ), <code>lt</code> ( $<$ ), <code>gt</code> ( $>$ ), or <code>eq</code> ( $=$ ). |
| <code>attempts</code>                              | Array of human score records used by the evaluator.                                                                                                                                                                          |
| <code>attempts[ ].id</code>                        | Identifier for the human attempt.                                                                                                                                                                                            |
| <code>attempts[ ].description</code>               | Required. Short method summary: optimizer, schedule, epochs, and any other key implementation choices. E.g., "AdamW with cosine annealing, 100 epochs"                                                                       |
| <code>attempts[ ].metrics[ ].&lt;metric&gt;</code> | One field per metric your evaluator emits (e.g., <code>top1_accuracy</code> , <code>top5_accuracy</code> ). Numeric, not string.                                                                                             |
| <code>attempts[ ].architecture</code>              | Required. The model architecture and any pretrained backbone used. E.g., "Pre-trained ResNet-50 fine-tuned on CIFAR-100"                                                                                                     |
| <code>attempts[ ].augmentations</code>             | Required. All training and data-augmentation enhancements applied. E.g., "Mixup and CutMix"                                                                                                                                  |

The metric field names must exactly match the keys returned in the `metrics` dictionary by `tests/checks.py`. If the evaluator emits `f1_score`, the field is `f1_score`. If it emits `top1_accuracy`, the field is `top1_accuracy`.

## How to produce your score

1. Run your oracle end-to-end through the task in your local Harbor sandbox
2. Capture the metric values returned by `tests/checks.py` and shown in the grading output
3. Write them into `human_scores.json` in the format consumed by `tests/pass_fail.py`.

## Common mistakes

- Omitting human score data required for the evaluator to calculate the final Pass/Fail Grade.
- Putting the file outside the task-level `tests/` directory.
- Storing scores as strings ("`0.816`") instead of numbers (`0.816`)
- Using metric names that do not match the keys returned by `tests/checks.py`

# **Milestones and Rubrics**

# Milestones and Rubrics

## (milestones\_and\_rubrics.json)

---

`milestones_and_rubrics.json` defines the task's milestones, rubric criteria, and rewards. `tests/rubric_score.py` uses it to calculate the Per-Milestone Scores and Overall Rubric Score. `tests/pass_fail.py` then uses those results with `tests/human_scores.json` to calculate the Binary Pass/Fail Grade.

## Where it lives

```
None
| | — tests/
| |   | — test.sh
| |   | — checks.py
| |   | — pass_fail.py
| |   | — rubric_score.py
| |   | — milestones_and_rubrics.json
| |   | — human_scores.json
```

## Requirements

Every task must have:

- At least 2 intermediate milestones
  - Mark load-bearing intermediate milestones as `critical: true`.  
Non-critical milestones may still contribute to the Rubric Score without independently forcing a Fail.

Every criteria must be deterministic.

## Criteria

Each criterion must be atomic, binary, verifiable, and weighted (positive or negative).

- Deterministic milestones: at least 1 criterion

The *Rubric Score* is the weighted sum of passing criteria from 0 to 1. `rubric_score.py` calculates the Per-Milestone Scores and Overall Rubric Score. `pass_fail.py` automatically generates the final pass/fail milestone and calculates the Binary Pass/Fail Grade.

## Example

json

```
JSON
{
  "task_id": "robust-ai-image-detection-deepshield",
  "milestones": [
    {
      "id": "data-ingestion",
      "name": "Data Ingestion and Provenance",
      "description": "Load the DeepShield subset, validate image paths, preserve provenance, and write normalized train/test manifests",
      "critical": true,
      "type": "deterministic", -- should always be deterministic
      "criteria": [
        {
          "id": "prepared-manifests-valid",
          "description": "prepared train/test CSVs exist with required columns, valid image paths, binary labels, fixed clean train split, and fixed social-platform test split",
          "weight": 2.0,
        },
        {
          "id": "dataset-summary-valid",
          "description": "dataset_summary.json reports split counts, label/platform/generator distributions, DOI, and license provenance",
          "weight": 1.0,
        }
      ]
    }
  ]
}
```

## Field reference

| Field                                 | Description                                                                                                                             |
|---------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| <code>task_id</code>                  | Identifier for the task.                                                                                                                |
| <code>milestones[].id</code>          | Stable kebab-case identifier.                                                                                                           |
| <code>milestones[].name</code>        | Name of the milestone                                                                                                                   |
| <code>milestones[].description</code> | Short description of what the milestone entails                                                                                         |
| <code>milestones[].type</code>        | <code>deterministic</code> (always)                                                                                                     |
| <code>milestones[].critical</code>    | Whether failure of this milestone forces the Binary Pass/Fail Grade to 0. Set to <code>true</code> for load-bearing stages of the task. |
| <code>milestones[].criteria</code>    | Array of binary criteria.                                                                                                               |
| <code>criteria[].id</code>            | Kebab-case identifier of the criterion                                                                                                  |
| <code>criteria[].description</code>   | Specific enough for a reviewer or judge model to apply consistently.                                                                    |
| <code>criteria[].weight</code>        | Weighted between [-5,5] depending on importance                                                                                         |

## Common mistakes

- Fewer than 2 intermediate milestones
- Criteria that combine multiple checks ("outputs are complete and correct")
- Vague criteria like "solution is good" without saying what that means
- Rubric criteria for requirements not present in `instruction.md`

# Test Directory Overview

# Test Directory Overview

## tests/ Directory

The tests/ directory is what turns a completed task run into a graded result. After the agent (or your oracle) finishes, the harness runs everything here to produce three numbers: Per-Milestone Scores, an Overall Rubric Score, and a Binary Pass/Fail Grade.

This page covers what each file is and how they interact; for guidance on designing a comprehensive suite, see the *Writing Tests* page. Six files do the work, and they have a strict division of labor: three files you never touch (the engine), one file where you write your task's logic, and two JSON files that configure how that logic is scored.

### How they fit together (the data flow)

1. `test.sh` is invoked by the harness and runs the two scoring scripts in order: `rubric_score.py`, then `pass_fail.py`.
2. `rubric_score.py` imports your `checks.py` and calls its `evaluate()` function exactly once.
3. `checks.py` runs every check you wrote and returns two things: a list of criteria (each one passed/failed) and a metrics dict (the agent's actual measured values, e.g. `{"auroc": 0.83}`).
4. `rubric_score.py` looks up the weight for each criterion and the critical flag for each milestone in `milestones_and_rubrics.json`, validates that every id matches, and computes the Per-Milestone Scores and Overall Rubric Score. It writes everything to an intermediate `rubric_result.json`.
5. `pass_fail.py` reads that intermediate file (your checks never run twice), compares the metrics against the pass thresholds in `human_scores.json`, generates the final pass/fail milestone from them, and computes the Binary Pass/Fail Grade.
6. `pass_fail.py` writes the final scores out as the reward.

So `checks.py` only ever decides: *did this criterion pass*, and *what was the measured value*. It never decides weighting or pass/fail; that's the harness's job, driven by the two JSON config files. This separation is deliberate: it keeps scoring logic in one audited place and lets reviewers reason about your rubric without reading your Python.

### rubric\_score.py (do not edit)

The first of two fixed, shared harness scripts. You should never modify it. It contains no task-specific logic. Its job is to import `checks.py`, call `evaluate()` once, validate your configuration, and compute the diagnostic scores:

- **Per-Milestone Scores:** the weighted sum of passing criteria within each milestone (0 to 1). A milestone passes when its score is 1.0, meaning every positive-weight criterion passed and no penalty criterion fired.
- **Overall Rubric Score:** the weighted sum of all passing criteria across the intermediate milestones (0 to 1). Diagnostic.

It also validates the wiring on every run: ids in `checks.py` must match `milestones_and_rubrics.json`, you need at least two intermediate milestones, and the file must not declare a final pass/fail milestone (that one is auto-generated). Any mismatch produces an immediate, named configuration error rather than a silently wrong score.

## pass\_fail.py (do not edit)

The second fixed harness script. You should never modify it. It never re-runs your checks; it reads what `rubric_score.py` produced and computes:

- **Binary Pass/Fail Grade:** 1 only if every critical intermediate milestone passes AND every primary metric meets its human-derived threshold; otherwise 0. This is the grade that matters.

It also generates the final pass/fail milestone automatically, one criterion per `pass_criteria` metric (for example, `auroc = 0.90 >= 0.80` reports as passed). You never declare this milestone and never write code for it; `pass_criteria` in `human_scores.json` is its single source of truth.

If you find yourself wanting to edit either harness script, that's a signal the logic belongs in `checks.py` (a new check) or in the JSON config (a weight, a critical flag, or a threshold) instead.

## checks.py

This is where all your task-specific verification lives. You write one check function per thing you want to validate: files exist, schemas are well-formed, predictions are valid probabilities, splits don't overlap, a metric was computed on the right data, and so on. A check passes by returning normally and fails by raising (the wrapper catches the exception and records the failure detail). For the principles behind a strong suite (testing behavior rather than grepping source, covering implicit requirements, and catching a bare-minimum solution) see the [Writing Tests](#) page.

Every placeholder check in the skeleton raises `NotImplementedError` until you replace it, so an unedited check can never pass silently. Delete the raise line only once you have implemented the check beneath it.

`checks.py` must expose an `evaluate(context)` function that returns:

- Two alignment rules are essential here:

- Every criterion `id` and `milestone_id` you return must exactly match an entry in `milestones_and_rubrics.json`. That file owns the weights and critical flags; `checks.py` only reports pass/fail.
- Every key in the `metrics` dict must exactly match a key in `human_scores.json`'s `pass_criteria`. The harness compares them.

**DO NOT** implement threshold comparisons in `checks.py`. Just read the agent's metric value and return it under `metrics`; the harness does the comparison against the threshold. Where possible, prefer recomputing the metric yourself from the agent's raw outputs (for example, scoring its predictions file against held-out labels) over trusting a self-reported metrics file. (You can still write *sanity* checks, e.g. "AUROC is above random chance," as ordinary criteria; just keep those well below the real human bar.)

## milestones\_and\_rubrics.json

This file defines your intermediate milestones (at least two) and the criteria inside each one.

- Each milestone carries a critical flag (`true` / `false`, defaults to `true` when omitted).
- Each criterion carries a weight: its contribution to the Rubric Score. Negative weights create penalty criteria.

The critical flag is the key knob, and it lives on the milestone, not the criterion. A milestone passes only when every one of its criteria passes (its score reaches 1.0), and the Binary Pass/Fail Grade drops to 0 the moment any `critical: true` milestone fails to pass. Non-critical milestones still contribute their criteria to the Rubric Score, but failing them won't fail the task. Mark the load-bearing stages critical (data preparation, model training) and leave quality-of-life stages (report depth) non-critical.

## human\_scores.json

This file defines the primary metric(s) that gate the Pass/Fail Grade, via the `pass_criteria` block. For each primary metric you specify the metric name and the threshold it must clear:

- Simple form (higher-is-better, uses `>=`): `"auroc": 0.80`
- Object form (lower-is-better or exact match): `"loss": {"threshold": 0.5, "comparison": "lte"}`

Supported comparisons are `gte`, `gt`, `lte`, `lt`, `eq`; the simple form defaults to `gte`. The metric names here must match the keys your `checks.py` returns under `metrics`. These same keys become the criteria of the auto-generated final pass/fail milestone.

You should also log the resulting metrics from your oracle as a human attempt here

## test.sh

A thin wrapper that the harness calls to kick everything off: it runs `rubric_score.py` and then `pass_fail.py`, and ensures a reward file is written even on failure (`pass_fail.py` always runs, so a crashed check still produces a graded result with the error attached). You generally leave it alone, editing only if you need to add a Python dependency for the verifier itself.

## Keeping the files in sync

The linkage between these files is by string identifiers:

- criterion `id` / `milestone_id` in `checks.py` ↔ entries in `milestones_and_rubrics.json`
- metric keys in `checks.py` `metrics` ↔ keys in `human_scores.json` `pass_criteria`

If any of these names disagree, the harness fails loudly: the run scores 0 and the result names exactly which ids are out of sync. You don't need agent outputs to check the wiring; running `tests/test.sh` at any point surfaces configuration errors immediately.

Your definition of done: run your oracle end-to-end, then run `tests/test.sh`, and confirm the reward is 1.0 (see the [Oracle Solution](#) and [Running Agents](#) pages). If the top human solution can't pass your own tests, the task isn't ready. Confirm the run produces every metric key your `pass_criteria` references, and that every criterion the harness expects is actually returned by `checks.py`. Reviewers check this same alignment against the [Submission Checklist](#).

# **Workflow B: Human Attempt**

# Workflow B: Human Attempt

---

In a human attempt, you're solving a task someone else created. You write a `solve.sh` that completes the full task, and your submitted solution becomes part of the human attempt pool

Your goal: solve every task as well as you can. Phoned-in attempts will visibly score lower than serious ones. The solutions you write must run without error to be accepted.

## What You'll Receive

You'll be assigned a task and given a zip containing the full task folder. Download the zip and open it.

```
None
├─ task_name/
│   ├── instruction.md
│   ├── task.toml
│   ├── solution/
│   │   └─ solve.sh
│   ├── environment/
│   │   ├── task_inputs/
│   │   └─ Dockerfile
│   └─ tests/
├─ test.sh
├─ checks.py
├─ rubric_score.py
├─ pass_fail.py
├─ milestones_and_rubrics.json
├─ human_scores.json
└─ harbor_config.yaml
```

## Steps

### 1. Read the instructions

Unzip the task and read `instruction.md`. This is the same prompt the AI agent will receive. It tells you what each milestone is, what outputs are expected, and any constraints.

## 2. Spin up the environment

None

```
stb harbor tasks start-env -p ./my-task-name -i
```

This drops you into the same Docker sandbox the agent will run in. You have access to everything in `environment/task_inputs/`.

## 3. Write your own Oracle Solution (`solve.sh`)

Write your `solution/solve.sh` so it captures everything needed to reproduce your solution end-to-end.

Your `solve.sh` **MUST** be your own original work. Do not copy, reuse, or adapt another/the provided oracle solution. This will result in your task being rejected!

Your `solve.sh` must:

- Run end-to-end without manual intervention
- Produce every output listed in `instruction.md`
- Be reproducible (running it twice produces consistent results)
- Stay within the task's `timeout_sec`
- Be written without LLM assistance

See the Oracle Solution page for detailed guidance on writing a solid [solve.sh](#).

## 4. Run it locally to make sure it works

None

```
stb harbor run -a oracle -e modal -p ./my-task-name
```

This runs your solutions through the task tests in order. Confirm that the entire task runs without errors.

## 5. Overwrite *ONLY* the human attempt `human_scores.json`

You should **overwrite the human attempt** `human_scores.json` with the results from your solution. Only update the `attempts` array. Do not touch the `task_id` and `pass_criteria`. This means that when you upload, there should only be one entry in `human_scores.json`.

## 6. Submit

Zip your completed task folder and upload on platform. Make sure your [solve.sh](#) replaces the existing one when you upload.

Also copy/paste the json from `human_scores.json` into the json field on the platform when prompted.

## Rules

- **No LLM assistance.** The whole point of human attempts is to establish a real human baseline. Using an LLM defeats the purpose and contaminates the dataset.
- **Stay within the timeouts.** Stay within the runtime limit. The task must complete within the `runtime_limits` declared in `task.toml`, no more than 4 hours total.
- **Solve every milestone.** Skipping a milestone means the chain breaks and the rest of your submission can't be evaluated.

# Evals

# Reviewing Eval Output

*Workflow A & B surface eval output directly on the submission when there is an issue.*

---

## What you'll see

When a submission fails verification or is flagged for revision, an optional code block appears on the submission populated with the results from that run. The code block contains the actual output from the run, including build errors, failing test output, oracle execution failures, or whatever else caused the verifier to flag the submission.

## How to use it during review

1. Check the code block first. If it contains text, the submission failed verification and the output tells you why. If the code block has no text, the submission clears the verifier and you can proceed with normal review.
2. Read the verifier output before requesting changes. Don't ask the contributor to re-run things you can already see failed, since the output is already there.
3. Distinguish task-level failures from contributor failures. A failing oracle, missing dependencies, or a broken Dockerfile is a contributor issue. A flaky verifier run or an infra hiccup is not, so flag those rather than rejecting the submission.
4. Cross-reference with the Submission Checklist. The verifier catches reproducibility, environment, and oracle issues. It does not catch instruction quality, difficulty calibration, or task uniqueness, so you still own those judgments.

## What the verifier is checking

For Workflow A submissions, the verifier runs the full oracle sequence: builds the Docker environment, executes `solve.sh`, runs `tests/test.sh`, which executes `rubric_score.py` followed by `pass_fail.py`, confirms a Binary Pass/Fail Grade of 1, and runs frontier model evaluation to assign difficulty. A failure at any validation stage will populate the code block.

For Workflow B submissions, the verifier runs the contributor's `solve.sh` against the existing task tests and confirms the task clears its pass/fail milestone.

## In-app static checks

Before a task can be submitted, the platform runs `static_checks`. These checks validate basic submission requirements, such as required files, task structure, and schema-level issues. Static checks must pass before submission can proceed.

## When the code block is missing but you expected one

Verifier runs usually take 20-30 minutes. If a submission is marked for revision but the code block is empty, flag it in Slack, [#ecs-otter-ml-coding](#), so the team can confirm.

# Running Agents

# Testing Your Task with Agents

---

Two kinds of agent runs validate your task before submission:

- **Oracle Agent** runs your own `solve.sh` to confirm your task is solvable and your tests behave correctly.
- **Model Agent** runs a frontier model against your task to confirm the difficulty is appropriate.

Run the Oracle Agent first. If your oracle doesn't pass, your task is broken and the model results won't tell you anything useful.

## Prerequisites

- **stb CLI installed.** You'll need the Snorkel CLI (`stb`) installed locally. See Quick Start or the CLI User Guide for installation steps.
- **API key for model runs.** Use the Snorkel CLI to self-generate and self-replenish your API key. If you hit limits or issues, post in the project Slack channel for help.
- **Modal installed.** You will need modal to access GPUs.

## Environment setup (model runs only)

Set the following environment variables:

Shell

```
export OPENAI_API_KEY=<your-portkey-api-key>
export OPENAI_BASE_URL=https://api.portkey.ai/v1
export ANTHROPIC_API_KEY=<your-portkey-api-key>
export ANTHROPIC_BASE_URL=https://api.portkey.ai/v1
```

**#Note - these are the same keys!**

Tip: add these to your `~/ .bashrc` or `~/ .zshrc` for persistence.

---

# Part 1: Oracle Agent

---

## What the Oracle Agent does

The Oracle Agent runs your `solve.sh` end to end against your task's grader. It exists to confirm that your oracle is reproducible, that the environment is correct, and that the rubric you wrote actually scores your solution as a pass.

The Oracle Agent:

1. Builds the Docker environment from `environment/Dockerfile`
2. Mounts everything in `environment/task_inputs/`
3. Executes `solution/solve.sh`
4. Runs `tests/test.sh`, which executes `tests/rubric_score.py` and then `tests/pass_fail.py` using `tests/checks.py`, `tests/milestones_and_rubrics.json`, and `tests/human_scores.json`
5. Confirms every critical intermediate milestone passes and every primary metric clears its threshold in the automatically generated final pass/fail milestone
6. Confirms the full run completes within `runtime_limits` in `task.toml` (no greater than 4 hours)

The oracle is run multiple times to confirm reproducibility. Every run must pass. A flaky oracle (passes once, fails the next) will be sent back for revision.

If the Oracle Agent can't complete your task, no model agent will. Oracle validation is the floor.

## Running the Oracle Agent

Shell

```
# Basic run
```

```
stb harbor run -a oracle -e modal -p <path-to-task>
```

## Debugging a failing oracle

Work through this in order.

## ***Step 1: Identify the failure***

**Read the error output carefully:**

- Did the `solve.sh` itself error out, or did it run but fail the tests?
- Did it pass tests but fail the final task-level grader?
- What file or command was involved?

## ***Step 2: Reproduce interactively***

**Drop into the container to debug step by step:**

```
Shell
stb harbor tasks start-env -p <path-to-task> -i
```

Inside the container, run the commands from your `solve.sh` one at a time to find the broken step.

## ***Step 3: Fix and re-run***

Update the relevant file, re-run the oracle, and repeat until all 3 runs pass.

# **Common failure modes**

## ***Solve script fails to run***

**If your `solve.sh` errors out:**

- Enter interactive mode and run commands manually
- Check for typos, missing dependencies, wrong paths, or permission issues
- Confirm any tools your script uses are installed in the `Dockerfile`

## ***Solve runs but tests fail***

If `solve.sh` finishes but `tests/test.sh` reports grading failures:

- Check the test output for the specific failed criterion
- Verify your solution actually produces the expected output (right file, right format, right location)
- Common causes: incorrect output format, missing files, off-by-one errors, edge cases your oracle doesn't handle

## Container won't build

If the Docker environment fails to build:

- Check `environment/Dockerfile` syntax
- Verify the base image is accessible
- Check that dependency installation commands work
- Try building manually:

Shell

```
cd <path-to-task>/environment  
docker build -t test .
```

---

# Part 2: Model Agent

---

After your oracle passes, run a frontier model against the task to confirm the task is fair.

## Available models

| Model            | Identifier                      |
|------------------|---------------------------------|
| Gemini 2.5 Flash | openai/@gemini/gemini-2.5-flash |

|                     |                                 |
|---------------------|---------------------------------|
| Gemini 3.5<br>Flash | openai/@gemini/gemini-3.5-flash |
|---------------------|---------------------------------|

Gemini 3.5 Flash (with the Gemini-CLI agent) is the official evaluation model. Use it as your primary signal. [Running model agents](#)

Replace `<path-to-task>` with your task directory.

### Gemini 3.5 Flash:

Shell

```
stb harbor run \  
-m openai/@gemini/gemini-3.5-flash \  
-e modal \  
-p <path-to-task> \  
--agent-kwarg disallow_web_tools=true
```

## Interpreting results

Each run ends in one of three ways:

- **Pass.** The agent completes every milestone and passed the final task-level grader.
- **Fail (good).** The agent failed for a legitimate reason: reasoning errors, missed edge cases, didn't grasp the complexity, hit a real bottleneck. This is what you want.
- **Fail (bad).** The agent failed because of issues with your task: ambiguous instructions, environment problems, broken tests, missing context. Your task needs revision.

Pay attention to *where* the agent fails at:

- Failing at milestone 1 every run usually points to a setup or instruction problem, not a difficulty problem

## Analyzing failures

When a model agent fails:

- Watch the terminal recording to see what the agent tried

- Check the analysis output for identified issues
- Read the debug pane to understand the failure point

Then ask: good failure or bad failure?

**Good failure reasons:**

- Agent made reasoning errors
- Agent missed subtle requirements
- Agent lacked the domain knowledge the task requires
- Multi-milestone complexity tripped up state-tracking

**Bad failure reasons:**

- Instructions were ambiguous
- The environment had issues (missing dependencies, wrong CUDA version, etc.)
- Tests were too strict or non-deterministic
- Required information was missing from `instruction.md`

---

## Best practices

- **Run the oracle early and often.** Don't wait until submission.
- **Fix oracle failures before running model agents.** A broken task gives meaningless model results.
- **Read all test output.** Understand which criteria pass and fail at each milestone, not just the top-level result.
- **Keep your environment minimal.** Easier to debug, faster to build.
- **Test against Gemini 3.5 Flash before submitting.** Catching a too-easy or too-hard task at this stage saves a review cycle.

# Submission Checklist

# Submission Checklist

---

## Task Structure

| Criteria                                                          | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | Severity |
|-------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| Every task contains the required top-level files and directories. | Every task must contain: <code>instruction.md</code> , <code>task.toml</code> , <code>solution/solve.sh</code> , <code>environment/</code> with <code>Dockerfile</code> and <code>task_inputs/</code> , and <code>tests/</code> with <code>test.sh</code> , <code>checks.py</code> , <code>rubric_score.py</code> , <code>pass_fail.py</code> , <code>milestones_and_rubrics.json</code> , <code>human_scores.json</code>                                                                                         | High     |
| Tasks do not contain unnecessary files.                           | No leftover job logs, stray <code>data/</code> folders (all environment data should be in <code>environment/task_inputs/</code> ), <code>pycache</code> , <code>.ipynb_checkpoints</code> , or other artifacts not part of the task structure. This also includes files with suspicious or suggestive names (e.g., <code>cheat.py</code> , <code>answers.txt</code> , <code>solution_backup.sh</code> , <code>backdoor.*</code> ) that imply the task may have been tampered with or contains hidden answer keys. | Low      |

## Instruction (`instruction.md`)

| Criteria                       | Description                                                                                                                                                                                                                                                                                                      | Severity |
|--------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| Instruction is well specified. | The instruction should be concise but well specified. The goal of the task should be clear and obvious to both a human and an agent. Watch for tasks that are mostly difficult because they contain too many edge cases or detailed conditions; if a task is hard mainly for that reason, it should be rejected. | High     |

|                                                                                                               |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |      |
|---------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|
| Instruction is interesting.                                                                                   | The task should not be so obscure or irrelevant that no group of ML practitioners would find it useful. Every task must be interesting, practical, or valuable in some way.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | High |
| Instruction does not provide hints on how to solve the problem.                                               | Tasks should represent realistic ML research/engineering requests. If <code>instruction.md</code> includes significant hints, rubric exposure, or step-by-step solution guidance, it is not representative of the style we want. Stating requirements and deliverables is fine; explaining how to implement them is not. This also includes anything that would let an agent satisfy the rubric without actually solving the task (e.g., exposing expected output values, embedding test cases, or revealing the rubric's pass/fail thresholds in the instruction).                                                                                        | High |
| Instruction is unique relative to previously submitted tasks.                                                 | The task must be noticeably different from tasks in previously accepted Otter submissions. Similarity search results are provided to help make this determination. A task is usually too similar if neither the dataset/setup nor the expected deliverables differ in a non-trivial way.                                                                                                                                                                                                                                                                                                                                                                   | High |
| Instruction uses absolute paths.                                                                              | Any referenced file or directory path in the instruction must be an absolute reference (e.g., <code>/app/data/</code> , <code>/app/output/model.pth</code> ).                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | High |
| Instruction lists every deliverable, output path, schema, target metric, and threshold enforced by the tests. | <p>Every output file or interface that the rubric or tests check must also be listed in <code>instruction.md</code>. For each deliverable, specify the absolute output path and the expected schema or content.</p> <p>For structured outputs, include the exact required field names, column names, nesting or table structure, wrapper or record format, and value types. If the tests require a specific file name, field name, column name, function name, CLI argument, or output structure, that requirement must be explicitly stated in the instruction. The agent should never need to infer hidden interface requirements from the verifier.</p> | High |
| Task covers the full ML lifecycle.                                                                            | The task must span a meaningful slice of the end-to-end ML lifecycle rather than a single isolated step — for example data collection/preprocessing, model training under realistic constraints, and evaluation/reporting. A strong task forces the agent through multiple connected stages where a failure                                                                                                                                                                                                                                                                                                                                                | High |

|                                                               |                                                                                                                                                                                                                                                                                  |        |
|---------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
|                                                               | at any stage costs the whole task, reflecting realistic ML research and engineering work.                                                                                                                                                                                        |        |
| <code>instruction.md</code> does not contain a canary string. | The instruction should not contain a canary string. This is usually a key indicator that an older task skeleton was used, and the canary gets passed as part of the prompt.                                                                                                      | Medium |
| <code>instruction.md</code> does not contain the task name.   | The task name should not appear in <code>instruction.md</code> . Many tasks have a comment in the first line with the name or a variant. Remove it.                                                                                                                              | Medium |
| Instruction reads as human-authored.                          | The instruction should not exhibit hallmarks of LLM-generated text, such as formulaic structure, vague hedging language, generic ML jargon without task-specific details, or unnatural transitions. It should read like a real engineer or researcher writing a genuine request. | Medium |

## Environment (environment/)

| Criteria                                                                               | Description                                                                                                                                                                                                      | Severity |
|----------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| Dockerfile and build scripts do not grab content from the web (other than packages).   | The environment should not rely on any content from the internet other than package dependencies. Any content that needs to be downloaded should be stored locally in <code>environment/task_inputs/</code> .    | High     |
| All package dependencies use pinned versions.                                          | Any dependencies installed must use pinned versions. This is high severity for <code>pip</code> , <code>conda</code> , and other language-level package managers.                                                | High     |
| Environment does not use context from outside the <code>environment/</code> directory. | All content in the environment should be contained within <code>environment/</code> . The build should not capture context outside of this directory.                                                            | High     |
| Environment does not contain solutions or ground-truth answers.                        | The environment must not contain any files that provide oracle solutions or ground truth used by tests. The oracle script belongs in <code>solution/</code> , and test artifacts belong in <code>tests/</code> . | High     |

|                                                       |                                                                                                                                                                                                                                                                                                                                                                                                                                               |      |
|-------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|
| Dockerfile does not execute any dangerous operations. | The Dockerfile should not require dangerous operations like running in <code>--privileged</code> mode, using <code>SYS_ADMIN</code> , <code>NET_ADMIN</code> , <code>SYS_MODULE</code> capabilities, or mounting <code>/var/run/docker.sock</code> .                                                                                                                                                                                          | High |
| Task is single-container, single-GPU.                 | Multi-container setups (e.g., <code>docker-compose</code> with separate database services) are not accepted. Multi-GPU configurations are not accepted.                                                                                                                                                                                                                                                                                       | High |
| Dataset and asset licensing is permissive.            | All datasets, model checkpoints, code, and other assets bundled in <code>environment/task_inputs/</code> must use licenses that permit commercial use and redistribution. Copyleft licenses (e.g., GPL on data), research-only licenses, and non-commercial licenses are not acceptable. The environment must also not contain proprietary data, internal company materials, or anything not cleared for external release.                    | High |
| Data is real, not obviously synthetic.                | The task's data must come from a genuine source (a published dataset, a competition, or authentically collected data), not values fabricated to fit the task. Obviously synthetic data — randomly generated arrays, toy distributions, or numbers manufactured so the intended solution trivially works — is not acceptable. Real data carries the missing values, label noise, and distribution quirks that make the ML challenge authentic. | High |
| Apt package versions are pinned.                      | Nice to have, especially for niche apt packages likely to have significant functional changes between versions. Most common apt packages are not required to be pinned.                                                                                                                                                                                                                                                                       | Low  |
| Base image versions are pinned (digest preferred).    | Nice to have, especially for niche images likely to have significant changes between versions. Most common base images are not required to have pinned versions.                                                                                                                                                                                                                                                                              | Low  |
| Avoids using heredocs in Dockerfile.                  | Preferred that Dockerfiles do not use heredoc formats like <code>cat &lt;&lt; EOF</code> or <code>RUN cat &gt; /app/script &lt;&lt;'EOF'</code> .                                                                                                                                                                                                                                                                                             | Low  |

## Oracle Solution (solution/solve.sh)

| Criteria                                                                    | Description                                                                                                                                                                                                                                                                                                    | Severity |
|-----------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| Task has a working oracle.                                                  | <code>solution/solve.sh</code> must run end-to-end without manual intervention and produce every output required by <code>instruction.md</code> .                                                                                                                                                              | High     |
| Oracle passes the rubric.                                                   | When the oracle is run, it must achieve a Binary Pass/Fail Grade of 1 — passing every critical milestone and meeting every primary-metric threshold in <code>tests/human_scores.json</code> . The oracle should also score well on the diagnostic Rubric Score, since it represents a top-tier human solution. | High     |
| Oracle passes consistently with no flakey behavior.                         | Oracle solutions should avoid behavior that causes inconsistent passes between runs. This includes unseeded randomization, latency-based operations that vary by hardware, and any other source of non-determinism.                                                                                            | High     |
| Oracle does not require internet access during solve time.                  | The oracle must not require internet access, including downloading packages or data. All dependencies should be installed by the environment, and all data should be bundled in <code>environment/task_inputs/</code> .                                                                                        | High     |
| Oracle is reflective of what <code>instruction.md</code> describes.         | The oracle must actually solve the problem outlined in <code>instruction.md</code> . It should cover every requirement in the instruction, not just those that are tested. The solution must be a real implementation, not a hardcoded answer to satisfy the tests.                                            | High     |
| Oracle fits within <code>runtime_limits</code> .                            | The full oracle run, end to end, must complete within the <code>runtime_limits</code> declared in <code>task.toml</code> , and no more than 4 hours total.                                                                                                                                                     | High     |
| Oracle uses <code>set -e</code> (or equivalent) so failures halt execution. | <code>solve.sh</code> should fail fast on any error so that earlier failures don't silently corrupt downstream steps.                                                                                                                                                                                          | Medium   |

## Rubric and Tests (tests/)

| Criteria                                                                 | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                 | Severity |
|--------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| Rubric has at least 2 milestones.                                        | <code>tests/milestones_and_rubrics.json</code> must define at least 2 milestones that together cover the task's stages. There is no separate final pass/fail milestone; The Binary Pass/Fail Grade is computed by <code>pass_fail.py</code> from the critical milestone results produced by <code>rubric_score.py</code> and the primary-metric thresholds in <code>human_scores.json</code> .                                                              | High     |
| All milestones are deterministic.                                        | No "subjective" milestones. No LLM-as-judge grading. Every milestone uses checks that produce the same result on the same inputs.                                                                                                                                                                                                                                                                                                                           | High     |
| Every milestone has at least 1 atomic, binary, deterministic criterion.  | Each criterion checks one thing, returns pass/fail, and produces the same result for the same inputs. Criteria requiring human judgment do not belong in the rubric.                                                                                                                                                                                                                                                                                        | High     |
| Load-bearing milestones are marked critical                              | The <code>critical</code> flag belongs at the milestone level in <code>milestones_and_rubrics.json</code> , not on individual criteria. Any milestone whose failure should force a Binary Pass/Fail Grade of 0 must be marked <code>critical: true</code> . Non-critical milestones still contribute to the diagnostic Rubric Score.                                                                                                                        | High     |
| Fixed scoring harnesses are not changed                                  | <code>tests/rubric_score.py</code> and <code>tests/pass_fail.py</code> are the fixed, shared scoring harnesses and must not be modified. Task-specific verification belongs in <code>tests/checks.py</code> ; criterion weights and milestone critical flags belong in <code>tests/milestones_and_rubrics.json</code> ; primary-metric thresholds belong in <code>tests/human_scores.json</code> . Modifying either fixed harness is grounds for rejection. | High     |
| Pass/Fail is gated by critical milestones and primary-metric thresholds. | The harness assigns a Binary Pass/Fail Grade of 1 only when every required criterion passes AND every primary metric meets its threshold. <code>checks.py</code> returns the agent's measured metric values under <code>metrics</code> ; the thresholds live in the <code>pass_criteria</code> block of                                                                                                                                                     | High     |

|                                                                                  |                                                                                                                                                                                                                                                                                                                               |        |
|----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
|                                                                                  | <code>tests/human_scores.json</code> . <code>checks.py</code> must not implement the threshold comparison itself — the harness does it.                                                                                                                                                                                       |        |
| Tests use the same logic for oracle and agent runs.                              | Tests must not contain conditional logic that causes the oracle to be validated differently than the agent's solution. Oracle and agent must be evaluated equivalently and fairly.                                                                                                                                            | High   |
| Tests are aligned with the instruction.                                          | Tests must check requirements that are explicitly or implicitly defined in <code>instruction.md</code> . Tests must not check for things outside what the instruction asks for.                                                                                                                                               | High   |
| Tests check correctness, not just format or file existence.                      | Tests must actually verify that an implemented solution is correct (e.g., accuracy thresholds, output values match expectations). They should not settle for checking only that a file exists or that an output has the right shape. A test suite that would pass any output that's roughly the right format is insufficient. | High   |
| Tests are rigorous without being fragile.                                        | Tests should reject incomplete, incorrect, or gameable solutions while accepting valid solutions that use different implementation approaches. Checks must not depend on exact source structure, incidental formatting, timing-sensitive behavior, or one expected solution path.                                             | High   |
| Tests do not grab content from the web (other than pinned package dependencies). | Test runners can install pinned dependencies needed only for verification, but tests must not rely on network content at evaluation time.                                                                                                                                                                                     | High   |
| Criteria weights are sensible.                                                   | Each criterion has a numeric weight (positive or negative). Weights should reflect the relative importance of the criterion to the milestone, not all default to 1.0.                                                                                                                                                         | Medium |
| Tests do not grep agent source code to verify behavior.                          | Tests should run the agent's outputs and check results, not pattern-match against the agent's source. Source-code grepping is brittle and easy to game.                                                                                                                                                                       | Medium |

## Human Scores (tests/human\_scores.json)

| Criteria                                                                        | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | Severity |
|---------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| <code>human_scores.json</code> contains every human attempt with full metadata. | The file must exist at <code>tests/human_scores.json</code> , be valid JSON readable by the harness, and contain an entry for every human attempt. Each entry must include: the attempt's score(s), architecture or approach metadata, any training and augmentation enhancements used, and a short method description.                                                                                                                                                                                                                                                                                                                                                       | High     |
| Primary metric(s) make sense and thresholds are reasonable.                     | The <code>pass_criteria</code> block in <code>tests/human_scores.json</code> must gate on primary metric(s) that genuinely make sense as the success measure for this task — the right metric for the problem type, not an irrelevant or trivially-gameable one — with thresholds that are sensible and calibrated against the recorded human attempts: clearable by a strong solution but not by a trivial or random one. Each <code>pass_criteria</code> key must match a metric key returned by <code>checks.py</code> , and each threshold should sit at or below what real human attempts achieved (using the correct comparison direction for lower-is-better metrics). | High     |

## Task Metadata (task.toml)

| Criteria                                                      | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | Severity |
|---------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| <code>task.toml</code> contains all required metadata fields. | Must fill in every required field: <code>runtime_limits</code> , <code>name</code> , <code>description</code> , <code>domain</code> , <code>subdomain</code> , <code>tags</code> , <code>source_dataset</code> , <code>source_url</code> , <code>source_reference_id</code> , <code>source_license</code> , <code>build_timeout_sec</code> , <code>workdir</code> , <code>os</code> , <code>cpus</code> , <code>memory_mb</code> , <code>storage_mb</code> , <code>gpus</code> , <code>gpu_types</code> , <code>allow_internet</code> , <code>timeout_sec</code> (agent), <code>timeout_sec</code> (verifier) | High     |

|                                                                                      |                                                                                                                                                                                                                                                                                                                      |        |
|--------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| <code>domain</code> and <code>subdomain</code> are valid choices.                    | Only the six valid domains and their listed subdomains are allowed. Domain and subdomain must be a valid pairing.                                                                                                                                                                                                    | High   |
| <code>domain</code> and <code>subdomain</code> actually match the task's content.    | The chosen domain and subdomain must align with what the task is actually doing (e.g., don't tag something as Image Segmentation when it's really Image Classification with masks).                                                                                                                                  | High   |
| <code>runtime_limits</code> is $\leq$ 4 hours.                                       | The total runtime budget must fit within the 4-hour cap.                                                                                                                                                                                                                                                             | High   |
| <code>[environment]</code><br><code>allow_internet = true</code><br>must be present. | EC must include the following in the task.toml:<br><br><code>[environment]</code><br><code>allow_internet = true</code>                                                                                                                                                                                              | High   |
| Mandatory license field is present for Task Creation.                                | For Task Creation (Workflow A), <code>source_license</code> must be filled out under the <code>[metadata]</code> block in <code>task.toml</code> IF a dataset is specified. It needs to have a permissive license (e.g., MIT, Apache-2.0, CC-BY-4.0).                                                                | High   |
| <code>tags</code> are 3 to 6 relevant ML keywords.                                   | Tags should describe relevant techniques, frameworks, or libraries. Be specific ( <code>lora</code> over <code>fine-tuning</code> , <code>graph-attention-network</code> over <code>gnn</code> ). Include the framework when it matters. Don't pad to 6; three good tags beat six vague ones. No marketing language. | Medium |

# Common Errors

# Common Errors

---

A running list of the most frequent issues that come up during Otter task review. If you see your task flagged for any of these, here's what they mean and how to fix them.

## Dependencies Not Pinned

Every package the environment installs must use an exact version. Floating versions, version ranges, and unpinned installs all break reproducibility because the same Dockerfile can produce different environments at different times.

### Wrong:

None

```
RUN pip install torch
RUN pip install torch>=2.0
RUN pip install transformers
```

### Right:

None

```
RUN pip install torch==2.3.0
RUN pip install transformers==4.40.0
```

This applies to:

- Every line of `pip install` in the Dockerfile
- Every line of `requirements.txt` if you use one
- The base image (pin with a digest using `image@sha256:...` rather than a floating tag)
- Apt packages (nice to have, especially for niche packages)

If your task uses an external repo, pin it to a specific commit so agents cannot pull a newer version that may contain solutions.

## Test Misalignment (Not Full Coverage)

Tests must cover what the `instruction.md` actually asks for, not just a shallow subset of it. The most common form of this is a test suite that only checks file existence or basic format and would pass any output that's roughly the right shape.

### Symptoms:

- Tests check that a model checkpoint was produced, but not whether the predictions make sense
- The `instruction.md` lists multiple requirements, but only one or two are tested
- A bare-minimum agent that produces empty or junk output still passes the tests

**Fix:** For each `instruction.md`, walk through every requirement and confirm at least one test criterion covers it. Then add criteria for critical implicit requirements that any reasonable solution must satisfy (no NaNs, predictions in valid ranges, splits don't overlap, metric was computed on the right data, etc.). The bar is: would your tests catch an agent that did the bare minimum to produce output without actually solving the problem?

## Undocumented Output Requirements

A task may describe the correct output content without defining the exact format the verifier expects. This is unfair when agents produce semantically correct results but fail because they could not guess undocumented field names, columns, nesting, filenames, record structures, or interface details.

Common examples include:

- Requiring specific field or column names that are not listed in `instruction.md`
- Expecting a particular nesting, table, wrapper, or record structure without documenting it
- Requiring a specific filename, path, function name, or CLI output format that was never requested
- Expecting a particular data type, such as integer IDs instead of strings, without stating it

**Fix:** Review every hardcoded requirement in `tests/checks.py` and ensure it is explicitly stated in `instruction.md`, including the absolute path, required names, structure, and expected value types.

**Rule of thumb:** *If a file name, field name, column name, function name, or output structure is hardcoded in the tests, its existence and expected form must be specified in the instruction.*

## Instructions Overly Structured / LLM-Written

`instruction.md` files should sound like a real engineer's one-shot prompt to a terminal agent. They should not read like API documentation or like ChatGPT wrote them.

**Common signs an instruction is overly structured or LLM-generated:**

- Heavy use of headers, sub-headers, and nested bullet lists
- Function signatures, exact file paths, or exact data structures spelled out
- Bold markers highlighting requirements that read like answer keys
- Numbered "Step 1, Step 2, Step 3" walkthroughs of how to solve the problem
- Polite filler ("You are an expert programmer. Your goal is to...")
- Repetitive phrasing or overly polished tone

**Fix:** rewrite in your own voice as if you were giving the task to a teammate over Slack. Give the agent the *what* (the requirements) but not the *how* (the solution path). Vary your style across tasks so the dataset captures realistic prompting variance.

A good gut check: read your `instruction.md` out loud. Does it sound like something a real engineer would write, or does it sound like documentation?

It is **essential** to avoid overly prescriptive requirements that make the task too easy by telling the model which approach, architecture, implementation steps, or internal structure to use instead of only describing the expected outcome.

See the Instruction Prompt Styling page for the full set of examples and anti-patterns.

# Resources

# Project Otter Resources

---

[Slack channel](#)

[Example Task](#)

[Task skeleton](#)

[Task Gallery](#)